

CS 236: Operating Systems Lab

Spring 2025-26

Lab 4: xv6 cometh!

xv6 is a working operating system designed to expose the core ideas behind modern OS design in a controlled and understandable way. Developed at MIT, it is a re-implementation of the classic **Unix Version 6**, redesigned for teaching purposes.

Unlike well established operating systems such as **Linux** or **Windows**, xv6 is intentionally **small, readable, and well-structured**, making it ideal for learning core operating system concepts without the complexity of millions of lines of code.

To install xv6 on Ubuntu, look up details described in **Appendix A**.

The Holy Grail for xv6

- [xv6-risc book](#)
- [xv6-risc src booklet](#)

The xv6 source tree is organized to mirror OS subsystems:

- `kernel/` → process management, memory, file system, traps
- `user/` → user programs
- `mkfs/` → file system creation tool

Each directory corresponds to a logical OS component, making exploration and modification systematic. **You will mostly interact with `kernel/` and `user/` directory.**

User (space) Programs

xv6 is **intentionally simple** by design. Unlike modern operating systems such as Linux or Windows, xv6 **does not provide a compiler or development tools inside the OS**. e.g., shell based tools, gcc, editor etc.

This means that commands like: `gcc hello.c`
will not work inside xv6

Instead, xv6 follows a **static build model**: all user programs must be **compiled on the host machine** before the operating system boots.

How user programs are setup in xv6

1. Any user-space program you want to run must be written in the **user/** directory.
2. These programs are compiled **along with the kernel** during the xv6 build process.
3. After compilation, a **file system image** is created using `mkfs`.
4. This file system image contains:
 - o All compiled programs
 - o Any static files (e.g., `.txt` files)
5. QEMU loads this image when xv6 boots, making the programs available in the xv6 shell.

As a result, **new programs cannot be added at runtime**, they must be included **before booting xv6**.

6. Note that QEMU starts with two image files — **xv6.img** (the xv6 kernel) and **fs.img** (the root file system with all user files).

IMPORTANT

To ensure your programs in the **user/** directory are included in compilation, you need to modify the `Makefile`

- To add a program: Locate the `UPROGS` variable in the Makefile and add your user program by appending `$U/_<filename>\`
- To add static files (like `.txt`): Add them directly to the file system image

```
fs.img: mkfs/mkfs README sample.txt $(UPROGS)
mkfs/mkfs fs.img README sample.txt $(UPROGS)
```

Any changes in any of the user files need to be followed by a clean, compile and build, followed by executing xv6.

0. starting from the start

Find the xv6 directory on your local machine and try the following —

Start xv6:

```
Shell
$ cd xv6-riscv
$ make
$ make qemu
...
xv6 kernel is booting
hart 1 starting
hart 2 starting
init: starting sh
$
```

Try a few commands:

```
Shell
$ ls
$ cat README
$ echo hello
```

Exit QEMU using:

```
Shell
Ctrl + A then X
```

1. fork + exec + cat

In this part we will write a **user-space program p1.c** in xv6 —

- The program will accept a file name as a command line argument.
- The parent creates a **child process** and waits for the child to terminate. After the child terminates, the parent process prints its and its child's pid.
- The child process will print its pid and the file content. To print the file content use the **exec** system call to run the **cat** command in a suitable environment.

Hints:

- **cat** always reads input from file descriptor 0. Open the file on this descriptor to set up the environment.
- Make sure to include all required files (user program and data files) in the **Makefile**.
- A sample userspace program **q1.c** is provided. You can start by modifying this file.

Expected Output:

```
Shell
$ p1
Usage: p1 <filename>
$ p1 sample1.txt
[child]: my_pid: 5, file_content: hello cs236!
[parent]: my_pid: 4, child_pid: 5
$ p1 sample2.txt
[child]: my_pid: 7, file_content: hello from sample2!
[parent]: my_pid: 6, child_pid: 7
```

Constraints:

- No kernel modifications are allowed.
- File contents must be printed using **cat**.

System Calls

Several source files of xv6 are related to implementation of the system call mechanism and the system call interface.

Following is a list, details of which are needed for understanding xv6's implementation of the system call mechanism.

A keyword list of riscv ISA and xv6 functions and variables is listed in Appendix B.

- **user/user.h**
Contains **system call function declarations**. Any new syscall must be declared here so user programs can use/invoke it.
- **kernel/syscall.h**
Defines **system call numbers**. A new system call must be assigned a unique number here.
- **kernel/syscall.c**
Contains the **system call dispatch table**. This maps syscall numbers to kernel handler functions.
- **kernel/sysproc.c**
Most **process-related system calls** like fork, exit, etc., live here. Any new implementation of any process-related system calls can go here.
- **kernel/proc.h**
IMPORTANT - Declaration of **crucial process-related data structures** like PCB and process states is here.
- **kernel/proc.c**
The **heart of process management**. Contains almost everything that defines a process's **lifecycle** and **behavior**. You will get several examples of functions traversing/using the process list.
- **kernel/syscall.c**
Contains functions used to **safely extract syscall parameters passed from user space**.
- **user/usys.pl**
Every system call needs a **user-space wrapper function**. Produces **usys.S**, which contains assembly code that places syscall number in a register and executes **ecall**.
- **kernel/defs.h**
Contains **forward declarations for kernel functions** shared across multiple kernel files.

All or most of these files will have to be used/updated to implement/modify system calls.

Any changes in any of the kernel files need to be followed by a clean, compile and build, followed by executing xv6.

2. announcing newborn processes!

Like Linux, in xv6, the `fork()` system call is used to create a new process by duplicating the calling process. **Modify** the xv6 kernel implementation of the `fork()` system call so that **it will announce and print the pid of the new process**.

Write a user space program **p2.c** that will call the **fork** system call and print its pid and the child's pid in the parent process.

Hint:

- Locate the kernel implementation of the `fork()` system call in `kernel/`.

Expected Output:

When a process calls `fork()`, the xv6 console should display a message similar to —

```
Shell
$ p2
fork: created child with pid: 3
fork: created child with pid: 4
[parent]: my_pid: 3, child_pid: 4
```

Constraints:

- Do not change the behavior or return value of `fork()`
- The modification must be done **inside the kernel**, not in user space

3. knock, knock!

xv6 allows user programs to request services from the kernel using system calls. For this question, we will **add a new system call**, `knockknock()`, to xv6 that performs a simple task. The system call should print the pid of the calling process and the message “**Who’s there?**” to the xv6 console when invoked from a user program.

Hints:

- You need to modify some files mentioned above.

Expected Output:

You are given a testfile **p3.c** to test your implementation, the output should look like:

```
Shell
$ p3
```

```
[pid 3] Who's there?  
$ p3  
[pid 4] Who's there?
```

Constraints:

- All printing must be done **inside the kernel** in the **knockknock** system call.

4. peeking into the process table

xv6 maintains a process information table inside the kernel to track all processes in the system. Each process can be in different states, such as **UNUSED**, **SLEEPING**, **RUNNABLE**, etc.

For this question, we will **add a new system call** `getProcessStates()` that **prints the PID and state of every process whose state is not UNUSED by peeking in the process table of the kernel.**

Hints:

- Identify and locate the kernel data structure that stores per-process information in `proc.h`
- Refer to `allocproc()` in `proc.c` to understand how to iterate through the processes.

Expected Output:

You are given a testfile **p4.c** to test your implementation of `getProcessStates()`. After executing it, the xv6 console should display messages similar to:

```
Shell  
$ p4  
Parent pid: 3  
Process 1 was found in SLEEPING state  
Process 2 was found in SLEEPING state  
Process 3 was found in RUNNING state  
Process 4 was found in SLEEPING state  
Process 5 was found in SLEEPING state  
Process 6 was found in SLEEPING state  
  
=== Process states after exiting ===  
Process 1 was found in SLEEPING state  
Process 2 was found in SLEEPING state
```

```
Process 3 was found in RUNNING state
Process 4 was found in ZOMBIE state
Process 5 was found in ZOMBIE state
Process 6 was found in ZOMBIE state

=== Process states reaping ===
Process 1 was found in SLEEPING state
Process 2 was found in SLEEPING state
Process 3 was found in RUNNING state
```

Constraints:

- **Do not modify any internal data structure or kernel implementations.**
- Printing must be done from inside the kernel.
- The system call should not alter any process behavior.
- **Proper synchronization must be respected while accessing the processes.**

5. are you there, PID?

In the previous question, you implemented a system call that prints the state of all processes. In this question, you will **extend this idea** by creating a **parameterized system call** named `areYouThere(int pid)`. This system call will check and print the status of the process with the given `pid`.

If the process with the given PID does not exist, the system call should print an appropriate message.

Hints:

- Identify and locate the kernel data structure that stores per-process information in `proc.h`
- Refer to `allocproc()` in `proc.c` to understand how to iterate through the processes.
- Refer to functions defined in `sysproc.c` to understand how to accept function arguments for a system call.

Expected Output:

You are given a testfile `p5.c` to test your implementation of `areYouThere(pid)`. After executing it, the xv6 console should display messages similar to:


```
Shell
$ p5
Checking child pid 4
Process 4 is found in SLEEPING state

My PID is 3
Checking my own PID:
Process 3 is found in RUNNING state

Checking a non-existent PID (50)
50 has gone missing
```

Constraints:

- Printing must be done from inside the kernel.
- The system call should not alter any process behavior.
- **Proper synchronization must be respected while accessing the processes.**

6. counting forks

Currently, xv6 does not track how many child processes a process has created.

- Extend the PCB by adding a new field that tracks the **number of child processes created** by a process.
- Modify existing kernel logic so this field is **updated correctly**.
You need to modify existing system calls like `fork()` and `wait()`.
`fork()` will **increment** the child count variable in the PCB.
`wait()` will **decrement** the child count variable in the PCB.
- Add a new **system call with no arguments** named `getChildCount()` that returns the number of children created by the calling process.

Hints:

- Everything you did till now :)

Expected Output:

You are given a testfile `p6.c` to test your modifications and implementation of `getChildCount()`. After executing it, the xv6 console should display messages similar to:

```
C/C++
```

```
$ p6
```

```
Initial child count: 0
```

```
Child count after 3 forks: 3
```

```
Child count after reaping all childrens: 0
```

```
Child count after 4th fork: 1
```

```
Child count after reaping 4th child: 0
```

Constraints:

- Printing must be done from inside the kernel.
- The system call should not alter any process behavior.
- **Proper synchronization must be maintained when updating PCB fields.**

7. xtrace

In Linux systems, there's a powerful debugging tool called `strace` that intercepts and records all system calls made by a process. When you run `strace ls`, the tool lists every system call that `ls` makes, including arguments and return values.

As part of this lab, we will implement **xtrace**, a system call tracing mechanism for xv6 that allows processes to record and display all system calls made during their execution.

Two system calls to implement —

(i) `xtrace_start()`

- Enables tracing for the **current** process
- Records all subsequent system calls (including `xtrace_start()` itself)
- Returns: 0 on success, -1 on failure

(ii) `xtrace_end()`

- Disables tracing for the current process
- Prints all recorded system calls in chronological order
- Format: `[syscall_no] returned [return_value]`
- Returns: Total number of calls logged, or -1 if tracing was not active

Requirements

- Store up to 100 system calls
- Only record syscalls after `strace_start()` is called
- **Child processes should not inherit tracing**

Design hints

We will need to maintain some per process data, hence we can add a field to the proc data structure. We will also have to make sure that this is not getting inherited during a fork.

We also need to make sure that the extra data we allocate for storing the outputs is freed once the `xtrace_end()` syscall is called.

Sample output

```
Shell
$ p7a
Syscall Trace for PID 3:
  Syscall 22 returned 0
  Syscall 4 returned 0
  Syscall 11 returned 3
$ p7b
Syscall Trace for PID 4:
  Syscall 22 returned 0
  Syscall 4 returned 0
  Syscall 11 returned 4
  Syscall 1 returned 5
  Syscall 3 returned 5
$ p7c
Syscall Trace for PID 6:
  Syscall 22 returned 0
  Syscall 4 returned 0
  Syscall 11 returned 6
  Syscall 1 returned 7
```

If `xtrace_end` has not been called before `exit`, assume it is implicitly called and perform the necessary actions.

Submission Guidelines

Submit all changed files, including the updated Makefile, in a folder named `<rollno>_lab4`

Appendix A

Follow these steps for installing xv6 on a Ubuntu-based Linux machine:

1. Prerequisites

- a. First, update your system:

sudo apt update

- b. Install the required packages:

**sudo apt install -y git build-essential qemu-system-riscv64
gcc-riscv64-unknown-elf**

- **build-essential**

For essential packages like libc, gcc, make, etc.

- **qemu-system-riscv64**

Emulates a RISC-V machine in software.

Your computer's real CPU speaks x86-64, but xv6 is written for RISC-V.

QEMU **pretends to be a RISC-V computer**, so the OS can run even though the hardware is different.

- **gcc-riscv64-unknown-elf**

It runs on your x86-64 machine but produces **RISC-V binaries**, not x86-64 ones.

We need this because xv6 must be compiled for the **CPU architecture that QEMU is emulating**.

2. Verify toolchain installation

riscv64-unknown-elf-gcc --version

qemu-system-riscv64 --version

If both commands work, your environment is ready.

3. Clone the xv6 RISC-V Repository

**git clone <https://github.com/mit-pdos/xv6-riscv.git>
cd xv6-riscv**

4. Build xv6

make

Run the command to build the OS. If compilation succeeds, you are ready to run xv6.

5. Run xv6 on QEMU

Start the xv6 operating system:

make qemu

Appendix B

A list of riscv ISA items and xv6 items that intersect with the xv6 system call implementation — For details of these and more look up Chapter 4 of the xv6 book.

C/C++

ecall

sret

stvec

sepc

scause

sstatus (SIE and SPP)

sscratch

satp

csrw

a0

TRAMPOLINE

TRAPFRAME

uservec

usertrap

userret

prepare_return

syscall

copyin

copyout

copyinstr

argint

argstr